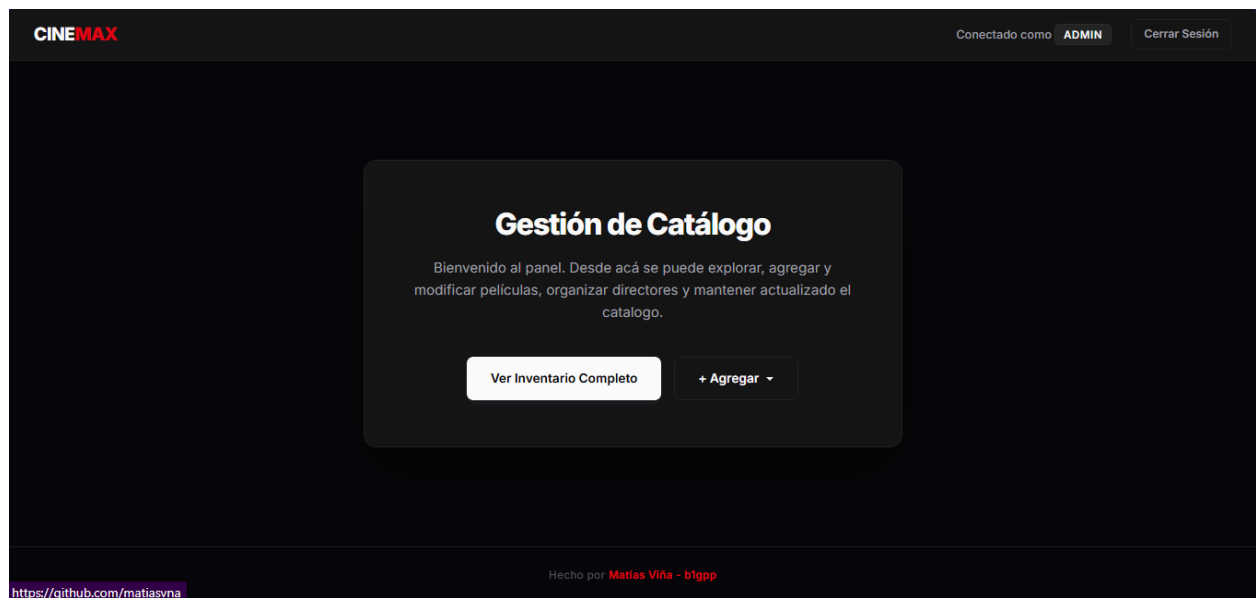


Laboratorio de Programación

Proyecto Final: CINEMAX

Software dedicado a catalogar películas



Realizado por el alumno Matías Viña

Universidad Nacional de la Patagonia Austral

Profesores: Reinaga Hector, Miranda Mirta, Oyarzo Fernanda



1. Proyecto: CINEMAX - Sistema de Gestión de Películas (Monolito Spring Boot)	3
Laboratorio de Programación	3
Stack de Tecnologías: Java 17, Spring Boot 3.3, Spring Data JPA, Hibernate, Spring Security 6, Thymeleaf, MySQL	3
1.1. Breve Descripción Teórica de las Tecnologías Utilizadas	3
2. Normalización del Modelo de Dominio y Mapeo Objeto-Relacional (JPA)	4
2.1. Corrección de la Cardinalidad de Géneros (De Restricción Unitaria a N:1)	4
2.2. Escalabilidad de la Cardinalidad de Actores (Relación N:M)	4
2.3. Desacople y Simplificación de Atributos Temporales (int año)	5
2.4. Incorporación de la Entidad Directores (Relación 1:N / N:M)	5
2.5. Creación de la Entidad Usuarios para Control de Acceso	5
3. Modernización de la Capa de Presentación y Refactorización UI/UX - Asistida con IA	5
3.1. Arquitectura de Hojas de Estilo Modulares	6
3.2. Soporte para Componentes Nativos del Agente de Usuario (color-scheme: dark)	6
3.3. Ampliación del Data Grid y Formateo Dinámico de Colecciones (películas.html)	6
4. Estabilización de Controladores y Ciclo de Vida HTTP (Spring MVC)	7
4.1. Sincronización del Data Binding y Tipos de Datos (Prevención de HTTP 500)	7
4.2. Estandarización de Endpoints y Acción del Formulario (Resolución de HTTP 404)	8
4.3. Inyección Contextual de Dependencias en el Modelo	8
5. Arquitectura de Seguridad, Autenticación y Control de Tráfico	9
5.1. Autenticación y Cifrado Criptográfico de Contraseñas	9
5.2. Mitigación de Fuerza Bruta mediante Bloqueo Proactivo de Cuentas (Account Lockout)	9
5.3. Limitación de Tasa y Filtrado Anti-Spam (FiltroPeticones.java)	9
5.4. Protección de Sesión, CSRF e Inmunidad frente a SQLi	10
6. Estrategia de Persistencia, Data Seeding y Portabilidad	10
7. Alcance del Sistema, Limitaciones Conocidas y Trabajo Futuro (Roadmap)	11
7.1. Flujo de Recuperación de Cuentas (Account Recovery)	11
7.2. Volatilidad del Registro de Limitación de Tasa	11
7.3. Sincronización de Expiración de Sesión (Client-Side)	11

1. Proyecto: CINEMAX - Sistema de Gestión de Películas

(Monolito Spring Boot)

Laboratorio de Programación

Stack de Tecnologías: Java 17, Spring Boot 3.3, Spring Data JPA, Hibernate, Spring Security 6, Thymeleaf, MySQL

1.1. Tecnologías Utilizadas

- **Spring Boot 3.3:** Marco de trabajo en Java diseñado para agilizar el desarrollo de componentes autónomos enfocados en entornos de producción. Ofrece un servidor web Tomcat integrado y mecanismos de autoconfiguración, evitando por completo la gestión mediante archivos XML adicionales.
- **Spring Data JPA & Hibernate:** La interfaz JPA constituye el estándar en Java destinado al mapeo objeto-relacional, siendo Hibernate su motor de implementación. Su integración posibilita la manipulación del esquema de datos utilizando clases de la aplicación en lugar de sentencias SQL directas.
- **Spring Security 6:** Módulo de arquitectura de seguridad y autenticación sumamente adaptable para el ecosistema Spring. Se encarga del resguardo de endpoints, la administración de la sesión del usuario y la neutralización de ataques perimetrales como CSRF.
- **Thymeleaf:** Motor para el procesamiento de plantillas dinámicas en el servidor. Facilita la renderización de archivos HTML vinculando directamente las colecciones e información del modelo con la interfaz visual de la aplicación.
- **MySQL:** Motor de base de datos relacional orientado a garantizar el almacenamiento estructurado y la persistencia relacional de las entidades del proyecto.

2. Normalización del Modelo de Dominio y Mapeo Objeto-Relacional (JPA)

En la primera entrega del desarrollo, el modelo de datos presentaba anomalías de diseño conceptual que limitaban la flexibilidad del catálogo y rompían los principios elementales de normalización de bases de datos relacionales. La refactorización se centró en corregir las cardinalidades y optimizar los tipos de datos en la entidad principal `Pelicula.java`.

2.1. Corrección de la Cardinalidad de Géneros (De Restricción Unitaria a N:1)

Inicialmente, la relación entre películas y géneros estaba acoplada de manera restrictiva, asumiendo erróneamente un esquema donde un género quedaba ligado a una única instancia de película. En el dominio cinematográfico real, esta lógica es insostenible: múltiples películas pertenecen al mismo género (por ejemplo, numerosas obras comparten "Drama" o "Acción"), mientras que una película se categoriza bajo un género principal.

- Se reestructuró la relación en `Pelicula.java` implementando la anotación `@ManyToOne` sobre el atributo `Genero genero`, vinculada mediante `@JoinColumn(name = "genero_id")`.
 - **Impacto relacional:** En MySQL, la clave primaria de la tabla `generos` se propaga como clave foránea (FK) en la tabla `peliculas`. Esto desacopla el catálogo de géneros, permitiendo que la tabla `generos` funcione como una entidad de referencia estática e inmutable reutilizada por infinitos registros de películas.
 - **Integridad referencial:** Hibernate gestiona la persistencia evitando la duplicación de registros de género cada vez que se da de alta una nueva película.

2.2. Escalabilidad de la Cardinalidad de Actores (Relación N:M)

El diseño original manejaba el concepto de "protagonista" como un campo de texto plano o una asociación singular. Esto impedía reflejar la realidad del reparto de una película y limitaba las capacidades de consulta cruzada.

- Se implementó una relación de multiplicidad bidireccional `@ManyToMany` entre `Pelicula` y `Actor`:
 - Se definió la colección encapsulada `private List<Actor> actores = new ArrayList<>();`.
 - Se configuró explícitamente la tabla asociativa mediante `@JoinTable(name = "pelicula_actor", joinColumns = @JoinColumn(name = "pelicula_id"), inverseJoinColumns = @JoinColumn(name = "actor_id"))`.
 - **Justificación técnica:** Una película cuenta con un elenco diverso, y un actor participa en múltiples producciones. La creación de la tabla intermedia `pelicula_actor` descompone la relación muchos a muchos en dos relaciones uno a muchos (1:N), cumpliendo con la Tercera Forma Normal (3FN) y evitando redundancias masivas en la base de datos.

2.3. Desacople y Simplificación de Atributos Temporales (**int año**)

- Durante el desarrollo se evaluó el uso de tipos temporales completos como `java.time.LocalDate` para la fecha de estreno. Sin embargo, en el contexto del negocio de un videoclub o catálogo general, el dato analítico de interés suele ser exclusivamente el año de lanzamiento y no el día calendario exacto.
 - **Resolución:** Se definió la propiedad `private int año`; mapeada como una columna numérica entera no nula (`nullable = false`).
 - **Motivo:** El uso de un tipo primitivo previene discrepancias de parseo entre formatos de fecha internacionales (ISO vs regional), elimina la necesidad de lidiar con desfases de zonas horarias (*timezones*) en el servidor Tomcat y simplifica la validación tanto en el motor de plantillas como en la base de datos –MODIFICAR

2.4. Incorporación de la Entidad Directores (Relación 1:N / N:M)

Atendiendo a las exigencias funcionales de la aplicación, se procedió a incorporar la estructura relacional y la clase persistente `Director` (definida mediante los atributos ID y Nombre).

- **Implementación:** Dado que un director puede dirigir múltiples películas, pero (en este modelo corto) una película suele tener un director principal, se optó por una relación `@ManyToOne` desde la entidad `Pelicula` hacía `Director`.
- **Impacto:** Esto genera una clave foránea `director_id` en la tabla de películas, garantizando la integridad de los datos y permitiendo filtrar el catálogo por director de manera eficiente.

2.5. Creación de la Entidad Usuarios para Control de Acceso

Para soportar la capa de seguridad, se creó la tabla `usuarios` mapeada a través de la entidad `Usuario.java`.

- **Estructura:** La tabla almacena el ID, el nombre de usuario (username), la contraseña cifrada (password) y campos de estado como `cuentaBloqueada` para la auditoría de seguridad.
- **Objetivo:** Esta tabla es la fuente de verdad que consulta Spring Security mediante el `UsuarioServicio` para autenticar los ingresos al sistema y gestionar las políticas de bloqueo por fuerza bruta.
-

3. Modernización de la Capa de Presentación y Refactorización UI/UX - Asistida con IA

El sistema evolucionó desde vistas rudimentarias basadas en estilos por defecto hacia una interfaz inmersiva y moderna bajo el paradigma **Dark Theme**, orientada a mejorar la usabilidad, reducir la fatiga visual y modernizar a los estándares utilizados en la actualidad

3.1. Arquitectura de Hojas de Estilo Modulares

- Para evitar colisiones de reglas y sobreescrituras descontroladas en el DOM, se abandonaron las plantillas globales monolíticas y se dividió el CSS en componentes modulares por recomendación del docente evaluador, alojados en `src/main/resources/static/css/` (`peliculas.css`, `aggPelicula.css`, `login.css`).

- **Gama cromática y contraste:** Se estableció una paleta basada en tonos carbón profundo combinada con acentos en rojo institucional (estilo cine, netflix) y bordes sutiles en, garantizando los estándares de contraste del consorcio W3C para accesibilidad web.
- **Contención del Layout:** Se aplicaron directivas `max-width` en los contenedores principales (500px para el formulario de alta y 1200px para el catálogo) acompañadas de márgenes automáticos (`margin: 0 auto`), evitando que los campos se extiendan de forma antiestética en monitores panorámicos de alta resolución.

3.2. Soporte para Componentes Nativos del Agente de Usuario (`color-scheme: dark`)

- Uno de los problemas más notorios en interfaces oscuras construidas con HTML5 ocurre con los elementos de formulario nativos del sistema operativo o del navegador (tales como calendarios, menús desplegables `<select>` y barras de desplazamiento). Por defecto, los navegadores renderizan el popup del calendario en un blanco brillante que desentona violentamente con la estética general.

- **Solución técnica:** Se utilizó la propiedad CSS estandarizada `color-scheme: dark;` en las clases de control (`.custom-input`). Esto enseña directamente al motor de renderizado del navegador (Blink, Gecko) para que dibuje los widgets internos del sistema operativo en su variante oscura nativa.

3.3. Ampliación del Data Grid y Formateo Dinámico de Colecciones (`peliculas.html`)

El catálogo visual de películas omitía datos críticos que hacían que la grilla pareciera vacía o incompleta en comparación con los datos persistidos en el modelo.

- **Incorporación de columnas:** Se agregaron al encabezado `<thead>` y al cuerpo `<tbody>` las columnas de **Año** y **Actores**.
- **Iteración y Concatenación Mediante `iterStat`:** Mostrar una colección `@ManyToMany` en una celda de tabla suele resultar en la impresión cruda de objetos (`[Actor@4f3c, Actor@1a2b]`) o en la ruptura visual de la grilla con saltos de línea innecesarios. Para resolverlo de forma limpia, se utilizó la variable de estado de iteración de Thymeleaf:

```
propiedad iterStat - Matias Viña

1 <span th:each="actor, iterStat :
  ${pelicula.actores}"
2     th:text="${actor.nombre} +
  (!${iterStat.last} ? ', ' : '')">
3 </span>
```

La propiedad `iterStat.last` evalúa booleanamente si el puntero se encuentra en el último elemento de la colección. Si no lo está, concatena automáticamente una coma y un espacio; si es el último, omite el carácter, logrando un formato de lectura natural ("*Lionel Messi, Diego Maradona*") en una única línea.

4. Estabilización de Controladores y Ciclo de Vida HTTP (Spring MVC)

La interacción entre las vistas generadas por Thymeleaf y el despachador de peticiones (`DispatcherServlet`) de Spring presentaba fallas funcionales críticas que interrumpían el flujo de navegación básico.

4.1. Sincronización del Data Binding y Tipos de Datos (Prevención de HTTP 500)

- Al intentar renderizar la pantalla de creación de películas (`/peliculas/new`), el servidor arrojaba recurrentemente una excepción interna `TemplateInputException` (HTTP 500).
 - **Problema:** El motor Thymeleaf intentaba enlazar variables de la vista con propiedades inexistentes en la clase Java. En el HTML se solicitaba un campo `*{fecha}` (de tipo fecha completa) mientras que el modelo requería `*{año}` (entero), y se intentaba bindear un único objeto `*{actor}` sobre una propiedad que en realidad era una lista `List<Actor> actores`.
 - **Resolución:** Se corrigieron los atributos `th:field` del formulario haciéndolos coincidir de manera estricta con la estructura de `Pelicula.java`. Además, para la selección de actores se habilitó el atributo `multiple` en el elemento `<select>`, permitiendo que Spring MVC intercepte el array de identificadores enviados en el cuerpo del request y los instancie automáticamente como entidades `Actor` gestionadas por el `EntityManager`.

4.2. Estandarización de Endpoints y Acción del Formulario (Resolución de HTTP 404)

Al presionar el botón "Guardar Película", el sistema devolvía una pantalla de error HTTP 404 informando *No static resource peliculas/guardar*.

- **Problema de ubicación en carpetas:** En `PeliculaControlador.java`, el controlador poseía la anotación `@RequestMapping("/peliculas")` en el encabezado de clase y un método con `@PostMapping` genérico sin sub-ruta asignada. Por lo tanto, el backend solo escuchaba envíos dirigidos exactamente a `/peliculas`. Sin embargo, la etiqueta del formulario HTML definía explícitamente `th:action="@{/peliculas/guardar}"`. Al no coincidir el mapping, Spring Boot deducía que se estaba solicitando un archivo estático llamado "guardar", fallando en su resolución.
- **Resolución:** Se refactorizó la firma del método en el controlador incorporando la ruta explícita:



```
PostMapping - Matias Viña

1 @PostMapping("/guardar")
2 public String
   savePelicula(@ModelAttribute("pelicula") Pelicula
   pelicula) {
3     peliculaServ.save(pelicula);
4     return "redirect:/peliculas";
5 }
```

De esta forma, el framework ataja la petición POST, persiste el objeto a través de la capa de servicio y ejecuta el patrón de diseño web **Post/Redirect/Get (PRG)** hacia `/peliculas`, evitando reenvíos accidentales del formulario al refrescar el navegador

4.3. Contextualización de Dependencias en el Modelo

- Para que los formularios de selección desplegables de las vistas de creación y edición muestren información real, se requirió que el controlador añada en el objeto `org.springframework.ui.Model` las listas completas de entidades maestras:
- Se añade `GeneroServicio`, `ActorRepositorio` y `DirectorRepositorio` mediante la anotación `@Autowired`.
- En los métodos `@GetMapping("/new")` y `@GetMapping("/editar/{id}")`, se cargaron las colecciones completas (`findAll()`) bajo los atributos "generos", "actores" y "directores", asegurando que el DOM disponga de todos los IDs y nombres necesarios para armar las opciones antes de enviar el HTML al cliente.

5. Arquitectura de Seguridad, Autenticación y Control de Tráfico

Una vez estabilizado el funcionamiento básico de la persistencia y la interfaz, se desplegó un esquema integral de seguridad perimetral para blindar la aplicación frente a vulnerabilidades comunes del Top 10 de [OWASP](#).

5.1. Autenticación y Cifrado Criptográfico de Contraseñas

El acceso al sistema está resguardado por Spring Security 6, bloqueando el acceso a toda ruta sensible a cualquier usuario anónimo.

- **BCrypt Hashing:** Las contraseñas de los usuarios no se guardan en texto claro bajo ninguna circunstancia. Se utiliza la implementación `BCryptPasswordEncoder`, la cual aplica una función de derivación de claves basada en el cifrador Blowfish. Cada clave almacenada incorpora un *salt* aleatorio de 16 bytes que muta de forma única para cada usuario. Esto neutraliza ataques de diccionario precalculados y garantiza que, aun si dos administradores usan la misma clave, sus representaciones en la tabla `usuarios` sean completamente disímiles.

```
// 1. user 1 (admin)
if (usuarioRep.findByUsername(username: "admin").isEmpty()) {
    Usuario admin = new Usuario();
    admin.setUsername(username: "admin");
    admin.setPassword(passwordEncoder.encode("admin123"));
    usuarioRep.save(admin);
    System.out.println(x: ">>> Usuario 'admin' creado con éxito.");
}

// 1.1 usuario prof
if (usuarioRep.findByUsername(username: "profesor").isEmpty()) {
    Usuario profesor = new Usuario();
    profesor.setUsername(username: "profesor");
    profesor.setPassword(passwordEncoder.encode("unpa2026"));
    usuarioRep.save(profesor);
    System.out.println(x: ">>> Usuario 'profesor' creado con éxito.");
}
```

		id	password	username	cuenta_bloqueada	intentos_fallidos
☐	Editar	☐	Copiar	☐	Borrar	1
						\$2a\$10\$1/vJTyIV6PWxx/D.TgFHgevdH6FFaYr1y2ZEP7qd1Mj...
				admin	0	0
☐	Editar	☐	Copiar	☐	Borrar	2
						\$2a\$10\$MpTc8nLAXD8Oev03H2aqOeMhmkdY08rXQ6SvP2FtykW...
				profesor	0	0

5.2. Mitigación de Fuerza Bruta mediante Bloqueo Proactivo de Cuentas (Account Lockout)

- Para evitar que atacantes intenten descifrar credenciales mediante herramientas de fuerza bruta automatizadas, se acopló la lógica de seguridad con la base de datos a través del componente `EventosSeguridad.java`:
 - **Captura de Eventos:** Se escucha el evento de bajo nivel `AuthenticationFailureBadCredentialsEvent`.
 - **Umbral de Bloqueo:** Cada falla de autenticación incrementa un contador en la entidad `Usuario`. Si el usuario alcanza los 5 intentos fallidos consecutivos, el campo booleano `cuentaBloqueada` se establece en `true`.
 - **Respuesta del Framework:** Al quedar la cuenta marcada como bloqueada, Spring Security rechaza cualquier intento subsiguiente arrojando un `LockedException`, imposibilitando el acceso incluso si el atacante ingresa la contraseña legítima posteriormente. En un ingreso exitoso, el contador se reinicia a cero de inmediato para evitar bloqueos por errores humanos acumulados en el tiempo.

5.3. Limitación de Tasa y Filtrado Anti-Spam (`FiltroPeticones.java`)

- Para proteger al servidor de ataques de Denegación de Servicio en capa de aplicación (DoS L7) o envíos automatizados de formularios, se creó un filtro de servlets (`jakarta.servlet.Filter`) que audita las peticiones por dirección IP apoyándose en una estructura concurrente en memoria (`ConcurrentHashMap`).
 - **Problema inicial de falsos positivos:** En un principio, el filtro auditaba todas las solicitudes con un umbral de 500 ms. Al cargar una página, el navegador solicita los archivos `.css` e imágenes en paralelo casi al mismo milisegundo, provocando que el propio filtro bloqueara al usuario legítimo con un error HTTP 429 *Too Many Requests* y rompiera los estilos visuales.
 - **Refactorización:** Se aplicó una doble condición en el middleware:
 - **Filtrado por Verbo Mutacional:** El cómputo de tiempo entre peticiones se limitó exclusivamente a métodos `POST`, `PUT` y `DELETE`, que son las acciones que realmente consumen recursos de CPU y base de datos.
 - **Bypass de Recursos Estáticos:** Se excluyeron del filtro las rutas que comienzan con `/css/`, `/js/` o peticiones a recursos estáticos, permitiendo que la carga visual de la aplicación ocurra de manera instantánea y paralela sin penalizaciones de cuota.

5.4. Protección de Sesión, CSRF e Inmunidad frente a SQLi

- **Gestión de Inactividad:** En el archivo `application.properties` se definió `server.servlet.session.timeout=15m`. Al transcurrir 15 minutos sin interacción

de red, el Tomcat embebido destruye el `JSESSIONID`, mitigando el riesgo de secuestro de sesión (*Session Hijacking*) en terminales de trabajo abandonados.

- **Tokens Anti-CSRF:** Spring Security y Thymeleaf inyectan automáticamente un token criptográfico oculto en cada formulario que realiza mutaciones (`POST`), verificando que cada orden haya sido generada genuinamente desde el navegador del usuario autenticado en la sesión actual y no desde un sitio externo malicioso.
- **Inmunidad a Inyecciones SQL:** Mediante el uso estricto de Spring Data JPA, toda interacción con la base de datos se ejecuta a través del mecanismo de consultas preparadas (*Prepared Statements*) de JDBC. Los parámetros enviados en los formularios se tratan como datos literales y jamás se concatenan como código SQL ejecutable, anulando cualquier vector de ataque por inyección.

6. Estrategia de Persistencia, Data Seeding y Portabilidad

Para facilitar la portabilidad del proyecto y permitir que un docente o evaluador pueda ejecutar y corregir el sistema sin configuraciones manuales complejas, se integraron mecanismos de automatización en el arranque:

- **Autocreación de Base de Datos y Tablas:** La cadena de conexión JDBC en `application.properties` incluye el parámetro `createDatabaseIfNotExist=true`. En combinación con la propiedad `spring.jpa.hibernate.ddl-auto=update`, la aplicación crea por sí sola la base de datos `cine` en el MySQL local (puerto 3306) e infiere la estructura de tablas a partir de las anotaciones `@Entity` de Java.
- **Inicialización Controlada de Datos (Data Seeding):** La clase principal `TestApplication.java` implementa la interfaz `CommandLineRunner`. Al iniciar la aplicación, un algoritmo comprueba mediante sentencias `count() == 0` si las tablas de géneros, directores y usuarios están vacías. Si el sistema se ejecuta por primera vez, inserta automáticamente un conjunto de géneros, un elenco básico de directores y la cuenta de administrador inicial (`admin` con clave `admin123` encriptada en BCrypt), garantizando que el entorno esté operativo desde el primer despliegue sin requerir la ejecución de scripts SQL externos.

7. Alcance del Sistema, Limitaciones Conocidas y Trabajo Futuro (Roadmap)

El desarrollo de CINEMAX alcanzó su objetivo primario de garantizar una arquitectura robusta y un login funcional. No obstante, desde la perspectiva de la ingeniería de software, se reconoce que algunas de las barreras de seguridad implementadas carecen aún de los flujos secundarios necesarios para su administración autónoma. A continuación se documenta la deuda técnica asumida y visión a futuro para su óptimo desempeño:

7.1. Flujo de Recuperación de Cuentas (Account Recovery)

- **Limitación actual:** Si bien el sistema bloquea proactivamente las cuentas tras 5 intentos fallidos (Account Lockout), actualmente no existe un flujo automatizado de autoservicio para revertir este estado en la interfaz de usuario.
- **Trabajo futuro:** Implementar un servicio de recuperación mediante envío de enlaces de un solo uso (One-Time Links) por correo electrónico (SMTP) con expiración temporal, o en su defecto, desarrollar un panel de "Super Administrador" (`ROLE_SUPERADMIN`) que permita el desbloqueo manual. Por el momento, revertir un bloqueo requiere intervención directa del DBA sobre el motor MySQL.

7.2. Volatilidad del Registro de Limitación de Tasa

- **Limitación actual:** El middleware `FiltroPetitionen` mitiga ataques DoS utilizando una estructura en la memoria RAM del servidor. Al estar en memoria volátil, el historial de IPs atacantes se purga automáticamente ante cualquier reinicio del servicio Tomcat, no logrando guardar esas IPs maliciosas y permitiendo su rehuso.
- **Trabajo futuro:** Migrar el almacenamiento de auditoría de red hacia una base de datos en memoria (como Redis) o registrar las incidencias severas en una tabla de auditoría en MySQL. Esto permitirá mantener persistencia y realizar análisis forense de IPs maliciosas a largo plazo.

7.3. Sincronización de Expiración de Sesión (Client-Side)

- **Limitación actual:** El servidor invalida estrictamente la sesión tras 15 minutos de inactividad, pero el cliente (navegador) no es notificado de forma asíncrona. La interfaz gráfica permanece estática hasta que el usuario intenta realizar una nueva petición, topándose con un redireccionamiento abrupto.
- **Trabajo futuro:** Implementar un script de *polling* en JavaScript o una conexión por WebSockets que advierta al administrador mediante un modal visual un minuto antes de que su sesión expire, mejorando considerablemente la Experiencia de Usuario (UX)